



2026-2학기 데이터사이언스

3강. 클로드 코드의 이해 (2): 스킬 · MCP · 에이전트 · 플러그인

2026. 9. 21.

CLAUDE.md 가 마흔 줄이 되면?

- 지난주에 위키 규칙을 CLAUDE.md 에 적었다
- 그런데 "월별 요약 만드는 절차" 까지 거기 적으면 길어진다
- 데이터를 매번 손으로 내려받는 일도 남아 있다

클로드 코드를 넓히는 네 가지

무엇	넓혀 주는 것	한 줄로	비유 — 주방
① 스킬	정해진 절차	"이 작업은 이 순서로"	레시피 카드
② MCP	바깥 도구와 데이터	"직접 가서 가져온다"	주방 설비와 식자재 창고
③ 에이전트	일을 나눠 맡기기	"따로 시키고 결과만 받는다"	옆 조리대의 보조 요리사
④ 플러그인	앞의 셋을 묶어 배포	"남과 나눠 쓴다"	주방 전체를 담은 프랜차이즈 패키지

CLAUDE.md 가 "**항상 이렇게**" 라면, 오늘 배우는 넷은 "**필요할 때 이것을**"

① 스킬: 절차를 파일 하나로 묶는다

쓰는 이유

- 같은 지시를 반복해서 붙여 넣는 상황
- 절차를 CLAUDE.md 에 적으면 모든 대화에서 읽힘

만드는 방식

- 폴더 하나에 SKILL.md 하나 — 폴더 이름이 곧 명령 이름
- 앞머리에 name · description, 그 아래 본문에 절차
- 길어지면 scripts/ · references/ · assets/ 로 넣어 냄

특징

- description 이 언제 꺼낼지를 정함
- 본문은 꺼낼 때만 읽힘 — 길어도 평소에 비용 없음

스킬 하나의 일반적인 생김새

.claude/skills/인제스트/

```
SKILL.md          필수 — 이 파일 하나면 스킬이다
---
name: 인제스트
description: raw/ 에 넣은 자료를 위키로
              정리한다. 논문·강의노트를 넣을 때 쓴다.
---
1. wiki/index.md 를 먼저 읽는다
2. 무엇을 만들지 먼저 말한다
3. sources/ 와 topics/ 에 쓰고 index 를 갱신한다
scripts/          매번 똑같이 돌릴 코드
references/       필요할 때만 읽는 긴 문서
assets/           결과물에 쓰는 서식·이미지
```

아래 셋은 선택. description 은 늘, 본문은 꺼낼 때, 나머지는 필요할 때 읽힘

출처: anthropics/skills · skill-creator (2026. 9. 17. 확인)

① 스킬: CLAUDE.md 와 스킬을 가르는 기준

	CLAUDE.md	스킬
답는 것	사실과 규범	절차
언제 읽히나	그 폴더의 모든 대화	그 작업을 부를 때만
길어지면	매번 비용이 든다	비용이 들지 않는다
예	"면적 하한은 60m ² "	"월별 요약을 이 순서로"

이미 불어난 CLAUDE.md 는 손으로 가르지 않는다. `claude-md-management` 플러그인이 파일을 읽고 무엇이 규범이고 무엇이 절차인지 갈라 준다.

`/claude-md-management:claude-md-improver` 로 점검하고, `/claude-md-management:revise-claude-md` 로 이번 대화에서 정한 것을 반영한다.

① 스킬: 스킬을 만들어야 할 때

세 가지 신호

1 같은 설명을 두 번 이상 반복

매번 프롬프트에 같은 문단을 붙여 넣고 있음

2 CLAUDE.md 의 한 절이 절차로 자람

"이 순서로" 가 세 줄을 넘으면 스킬로 옮길 때

3 손으로 되풀이하는 자동화 대상

내려받기 · 이름 바꾸기 · 표로 옮기기

다만 **세 번 넘게 할 일에만** 만든다 — 규격화에도 품이 든다

처음부터 자동화하지 않는다

먼저 **손으로 반복해 본다**. 무엇이 반복인지는 몇 번 해 봐야 드러난다.

그다음 **찾는 일 자체를 클로드에게 시킨다.**

그대로 쓴다

"지금까지 나눈 대화와 커밋 기록을 훑어 줘. 내가 반복해서 같은 순서로 시킨 일이 있으면 묶어서 목록으로 보여 줘.

각 항목마다 몇 번 반복됐는지, 스킬로 만들 값이 있는지를 같이 판단해 주고, 가장 값이 큰 것부터 순서를 매겨 줘."

기록이 근거다. 머리로 고른 자동화 대상은 대개 빗나간다

① 스킬: 어디에 두느냐로 범위가 갈린다

위치	범위	부르는 이름
~/.claude/skills/<이름>/SKILL.md	내 모든 프로젝트	/이름
.claude/skills/<이름>/SKILL.md	이 프로젝트	/이름
<플러그인>/skills/<이름>/SKILL.md	플러그인이 켜진 곳	/플러그인:이름

스킬은 **두 가지 방식으로 불린다** — 내가 /이름 으로 직접 부르거나, 클로드가 description 을 보고 알아서 꺼낸다.

스킬 파일을 고치면 세션 중에도 반영된다. 새 스킬 폴더를 처음 만들 때만 재시작이 필요할 수 있다.

출처: 클로드 코드 공식 문서 code.claude.com/docs/en/skills (2026. 8. 27. 확인)

① 스킬: 스킬을 만드는 스킬 — skill-creator

스킬을 빈 파일에서 쓰지 않는다. **엔트로픽이 공개한** `skill-creator` 를 가져다 쓴다 — 스킬을 만드는 절차 자체가 스킬로 들어 있다.

가져오는 두 가지 방법

터미널 · 클로드 코드 안

플러그인으로 설치한다

/plugin → skill-creator

폴더째 내려받아 둔다

```
npx skills add https://github.com/anthropics/skills \
--skill skill-creator
```

한 번 두면 **부르지 않아도 스스로 꺼내진다** — `description` 에 "스킬을 만들 때·고칠 때" 가 적혀 있기 때문

스킬을 만들 때 — 그대로 쓴다

"인제스트 스킬을 만들어 줘. raw/ 에 넣은 논문·강의노트를 위키로 옮기는 절차야. CLAUDE.md 의 인제스트 규칙을 그대로 따르면 돼.

부르지 않아도 꺼내지게 description 을 쓰고, 시험 프롬프트 두세 개로 실제로 돌려서 보여 줘."

시키면 이 순서로 간다

- 1 무엇을 하는 스킬인지 되묻기
- 2 `SKILL.md` 초안 쓰기
- 3 시험 프롬프트 두세 개 돌리기
- 4 쓴 결과와 쓰지 않은 결과를 대조
- 5 고쳐 쓰고 다시 돌리기
- 6 `description` 다듬기 — 잘 꺼내지는지가 마지막 관문

② MCP: 바깥 도구를 쫓는 규칙

지금까지



MCP 로 붙이면



나르는 일이 없어진다 — 대신 확인할 일이 남는다

② MCP: 붙이는 법

손으로 붙일 때

터미널

원격 서버 (주소가 있는 경우)

```
claude mcp add --transport http \  
  <이름> <주소>
```

내 컴퓨터에서 실행하는 서버

```
claude mcp add <이름> -- <실행 명령>
```

범위 - 기본값은 local

```
--scope local      나 · 이 프로젝트에서만  
--scope project    .mcp.json 으로 저장소에 함께  
--scope user       내 모든 프로젝트
```

붙은 서버 확인

```
/mcp
```

주소만 주고 시킬 때

그대로 쓴다

"<https://github.com/chrisryugj/korean-law-mcp> — 이 MCP 를 이 프로젝트에 붙여 줘.

README 의 설치 방법을 그대로 따르고, 인증키는 어느 파일에도 적지 말고 환경변수로 넣어 줘. 다 되면 /mcp 로 확인해 줘."

클로드가 README 를 읽고 맞는 명령을 골라 실행한다. **명령을 외울 필요가 없다**

인증키는 `.mcp.json` 에 적지 않는다. 그 파일은 저장소에 올라간다

명령과 화면은 버전마다 달라진다. 막히면 공식 문서를 먼저 보고, 그다음 오류 메시지를 그대로 클로드에게 준다

출처: github.com/chrisryugj/korean-law-mcp (2026. 9. 17. 확인)

② MCP: 기타 유용한 MCP들

이름	무엇을 해 주나	어디에
QGIS	레이어 추가·제거 · 공간 처리 알고리즘 실행 · 지도 렌더링	<code>github.com/jjsantos01/qgis_mcp</code> 내 컴퓨터에서 도는 서버. QGIS 플러그인 + <code>uv</code> 필요
KOSIS	국가통계포털 통계표를 화면 없이 바로 조회	<code>kosismcp2026.vercel.app/api/mcp</code> 원격 · 시범 운영 · 인증키 없음
Playwright	웹 브라우저 조작 — 페이지 열기 · 입력 · 수집	<code>claude mcp add playwright npx @playwright/mcp@latest</code> 내 컴퓨터에서 도는 서버. Node.js 18 이상
Context7	라이브러리 공식 문서를 현재 버전 으로 찾아 읽음	<code>mcp.context7.com/mcp</code> 원격 · 인증키 없이도 동작

QGIS MCP 의 도구 목록에는 이런 것이 있다 — `execute_code` : "Run arbitrary Python code in QGIS from Claude. **Very powerful, but also be very cautious using this tool.**" 내 컴퓨터에서 임의의 코드를 돌릴 수 있다는 뜻이다. 공식 플러그인 저장소에 있고 코드가 공개돼 있다 — 그것을 확인하고 붙이는 것이다.

출처: 각 저장소·서비스 문서 (2026. 9. 17. 확인). 인용한 경고 문구는 `jjsantos01/qgis_mcp` README 원문

② MCP: 붙였다고 믿지 않는다

조심할 것	왜	어떻게
아무 서버나 붙이지 않음 받은 것을 검사함	MCP 서버는 내 자료를 읽고 바깥으로 보낼 수 있음 자동이 되면 이상해도 그대로 지나감 — 손으로 받을 때 는 눈으로 봤음	만든 사람과 공개된 코드를 먼저 확인 무엇을 보고 판정할지 미리 정해 둠
도구가 늘면 느려짐	붙인 서버의 도구 설명이 전부 컨텍스트에 들어감 — 서 버가 늘수록 매 대화가 무거워짐	쓰지 않는 서버는 꺼 둠 전역(user)과 프로젝트(project) 설치를 구분
바깥이 바뀔	기관이 표의 칸 이름이나 단위를 바꾸면 오류 없이 빈칸 이 넘어옴 — 화면이 멀쩡해 보여도 숫자가 비어 있음	받은 표의 자릿수·단위·빈칸을 사람이 확인

두 번째가 이 수업의 자리다 — **무엇을 검증할지는 사람이 정한다.**

출처: 클로드 코드 공식 문서 code.claude.com/docs/en/mcp (2026. 8. 27. 확인)

③ 에이전트: 따로 시키고 결과만 받는다

서브에이전트는 **자기 대화창을 따로 가진** 클로드다. 시킨 일을 그 안에서 하고 **요약만 돌려준다**.

쓰는 이유 셋

- **본 대화가 깨끗해짐** — 로그·검색 결과가 넘어오지 않음
- **권한을 좁힐 수 있음** — 검사역에게는 읽기만 줌
- 여럿을 **동시에** 시킬 수 있음

`.claude/agents/` 에 두면 이 프로젝트, `~/.claude/agents/` 에 두면 모든 프로젝트에서 쓴다.

`.claude/agents/wiki-checker.md`

```
---
name: wiki-checker
description: 위키 페이지가 규칙대로
             쓰였는지 검사한다. 출처·서식·연결을 본다.
             페이지를 쓰거나 고친 뒤에는 먼저 쓴다.
tools: Read, Grep
model: sonnet
---
너는 위키 검사역이다. 고치지 말고
무엇이 잘못됐는지만 보고한다.
1. 출처 없는 사실 문장이 있는지
2. frontmatter 네 줄이 다 있는지
3. index.md 에 올라갔는지
4. 링크가 실제 페이지를 가리키는지
```

`tools` 로 **쓸 수 있는 도구를 제한** — 검사역에게 쓰기 권한을 주지 않는 것이 요점

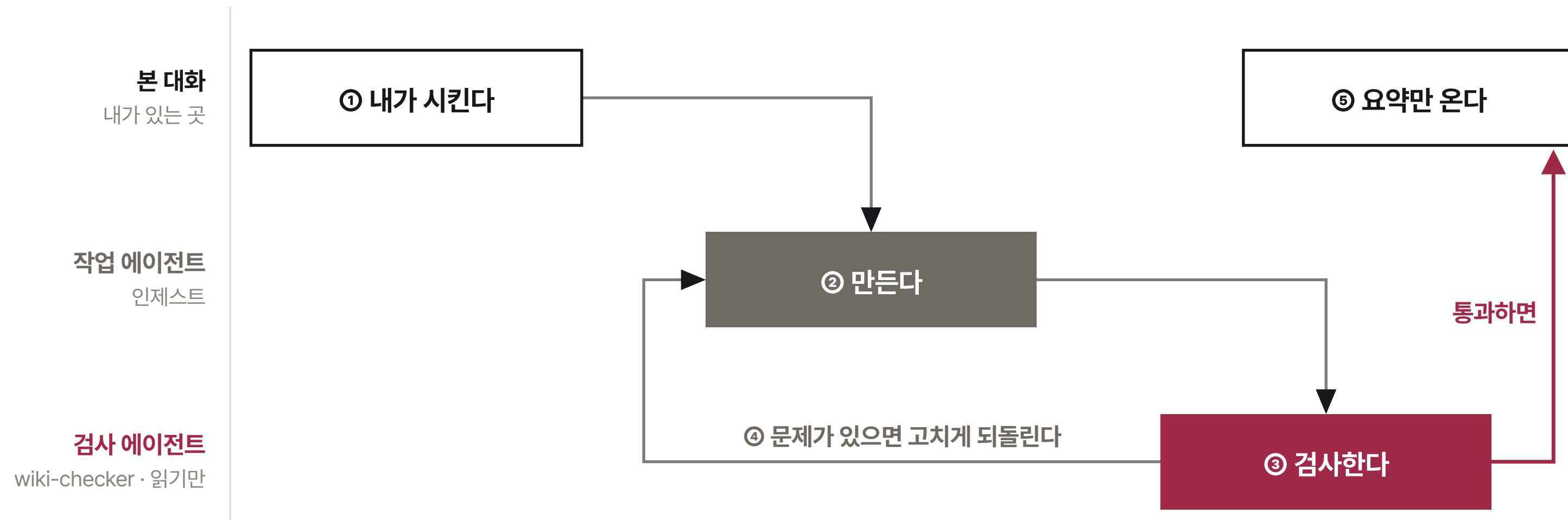
③ 에이전트: 부르는 네 가지 방법

방법	어떻게	이럴 때
클로드가 알아서	내가 부르지 않아도 넘김. <code>description</code> 에 <code>use proactively</code> 같은 문구를 넣어 둠	절차 안에 늘 끼는 일 — 쓸 때마다 하는 검사·조사
이름을 말함	" <u>wiki-checker</u> <u>에게 검사시켜 줘</u> " 라고 말함	이번 한 번만, 어느 에이전트인지 내가 알 때
@ 로 못 박음	<code>@agent-wiki-checker</code> 반드시 그것이 둠	검사·채점처럼 빠지면 안 되는 자리
세션 전체를 맡김	<code>claude --agent wiki-checker</code> 또는 <code>settings.json</code> 의 <code>agent</code>	이 세션 내내 그 역할만 할 때

평소에는 **첫째와 둘째**로 충분하다. 빠지면 안 되는 자리에만 @ 로 못 박는다.

출처: 클로드 코드 공식 문서 code.claude.com/docs/en/sub-agents (2026. 9. 17. 확인)

③ 에이전트: 누가 무엇을 하는가



본 대화에는 오간 로그가 아니라 ②~④ 의 요약만 남는다. 검사 에이전트에는 읽기 도구만 준다 — 만든 쪽과 보는 쪽을 나누는 것이 요점이다.

④ 플러그인: 묶어서 나눠 쓴다

앞의 셋 — 스킬 · MCP 설정 · 에이전트 — 을 **한 폴더에 묶은 것**이 플러그인이다.

혼자 쓸 때는 `.claude/` 로 충분하다. 남과 나눠 쓰거나 버전을 붙이려 할 때 플러그인으로 만든다.

설치하기

터미널 · 클로드 코드 안

`/plugin`

마켓플레이스에서 찾아 설치

`/reload-plugins`

설치 뒤 반영이 필요하다

앞에서 본 `claude-md-management` · `skill-creator` 가 그 예다.

만들기 — 폴더 구조

my-plugin/

`.claude-plugin/plugin.json`

이름 · 설명 · 버전

`skills/`

<이름>/SKILL.md

`agents/`

에이전트 정의

`hooks/`

hooks.json

`.mcp.json`

MCP 서버 설정

주의 — `.claude-plugin/` 안에는 `plugin.json` 만 넣는다. 나머지는 전부 플러그인 루트에 둔다.

시험해 보기

`claude --plugin-dir ./my-plugin`

출처: 클로드 코드 공식 문서 code.claude.com/docs/en/plugins (2026. 8. 27. 확인)

언제 쓰지 않을 것인가

한 번 하고 말 일에 스킬을 만들고 MCP 를 붙이고
루프를 짜면 손해다

→ 기준은 하나다. 반복하는가.

세 번 넘게 할 일이면 규격화가 이기고, 아니면 그냥 시키는 게 빠르다

실습

도구를 붙이고 · 붙이기 전과 견주기 — 약 1시간 30분

오늘 실습의 도착점

같은 질문에 **도구가 있을 때와 없을 때**가 어떻게 다른지를 화면으로 남긴다

시각	할 일	끝났다는 신호
0:00–0:15	① 먼저 물어본다 — 도구 없이	답을 그대로 저장했다
0:15–0:45	② MCP 둘을 붙인다	<code>/mcp</code> 에 둘 다 뜬다
0:45–1:15	③ 같은 질문을 다시 한다	두 답의 차이를 적었다
1:15–1:30	④ 첫 답의 인용을 검증한다	없는 조문이 × 로 찍힌다

①을 먼저 하는 이유가 있다. 도구를 붙인 뒤에는 클로드가 알아서 써 버려서 "없을 때의 답" 을 다시 만들 수 없다. 붙이기 전에 받아 두어야 비교가 된다.

시작 전 확인

클로드 코드가 켜지는 폴더 하나 · 인터넷이 되는 상태 · 법제처 인증키(21쪽에서 받는다)

① 먼저 물어본다 — 도구 없이 0:00–0:15

오늘의 사례 — 도시계획 실무에서 매번 나오는 질문이다

그대로 쓴다 — 아직 아무것도 붙이지 않은 상태에서

"청주시 제1종일반주거지역에 다세대주택을 지으려고 해. 건폐율과 용적률 상한이 얼마야?"

근거 조문을 법령 이름과 조 번호까지 적어 주고, 그 조문의 시행일도 같이 알려 줘."

- 1 받은 답을 한 글자도 고치지 말고 `답-도구미사용.md` 로 저장한다 5분
- 2 답 안의 **법령 이름 · 조 번호 · 숫자 · 시행일**에 표시를 해 둔다 — 나중에 이 네 가지를 대조한다 10분

✓ 확인 지점 — 답에 숫자가 들어 있다. 건폐율 몇 %, 용적률 몇 % 라고 말했을 것이다. **그 숫자가 어디서 왔는지**가 오늘의 쟁점이다.

이 질문을 고른 이유 — **법은 범위만 정하고 실제 숫자는 지자체 조례에 맡긴다.** 조례는 지자체마다 다르고 자주 바뀐다. 학습해서 아는 것과 찾아서 아는 것이 갈리는 자리다.

② MCP 둘을 붙인다 0:15-0:45

korean-law — 법제처 법령·판례·조례

0단계 · 인증키 발급 (무료)

open.law.go.kr 회원가입 → [Open API 사용 신청]

1단계 · 그대로 쓴다

"<https://github.com/chrisryugj/korean-law-mcp> — 이 MCP 를 붙여 줘.

README 의 설치 방법을 그대로 따라 줘. 인증키는 설치기가 물어 볼 때 내가 직접 넣을게 — 너는 어느 파일에도 적지 마. 다 되면 /mcp 로 확인해 줘."

클로드가 README 를 읽고 맞는 명령을 골라 실행한다. **명령을 외울 필요가 없다**

KOSIS — 국가통계포털

그대로 쓴다

"<https://kosismcp2026.vercel.app/api/mcp> — 이 원격 MCP 도 붙여 줘. 인증키는 없어.

붙인 뒤에 "청주시 총인구를 2020년부터 연도별로" 를 물어서 실제로 숫자가 오는지 확인해 줘."

✓ **확인 지점** — /mcp 에 korean-law 와 kosis 가 둘 다 ✓ **Connected** 로 뜬다.

Permission denied (publickey) 가 뜨면 — 설치기가 깃허브에 SSH 로 붙으려다 막힌 것이다. 그 오류 메시지를 그대로 클로드에게 준다 — 우회하는 한 줄을 넣고 다시 한다. 그래도 안 되면 다음 장으로 간다.

②-2 원격으로 붙일 때 인증키를 어디에 넣나

플러그인 설치가 막히면 **원격 서버로 바로 붙는 길**이 있다. 내 컴퓨터에 아무것도 깔지 않는다.

그대로 쓴다

"korean-law 를 원격으로 붙여 줘. 주소는 `https://mcp.gomdori.app/law` 이고 내 인증키는 헤더로 넘기고 싶어.

인증키를 파일에 적지 말고, 범위는 이 프로젝트 안에서만 쓰이게 해 줘."

클로드가 실행하는 명령

```
claude mcp add --transport http korean-law \
https://mcp.gomdori.app/law \
--header "apikey: 내인증키"
```

키를 넣는 자리 셋

어디에	언제 쓰나
<code>--header</code> apikey: 내키	원격으로 붙일 때. 주소와 키가 분리돼 화면 기록에 덜 남는다
주소 끝에 <code>?oc=내키</code>	웹.앱 클로드의 커넥터. 키가 주소에 붙어 있어 그대로 복사된다
<code>LAW_OC</code> 환경변수	내 컴퓨터에 직접 깔아 쓸 때

범위를 먼저 본다. `--scope project` 로 붙이면 설정이 `.mcp.json` 에 적혀 저장소에 함께 올라간다. 키가 든 설정은 기본값인 `local` 로 둔다 — 내 컴퓨터에 만 남는다.

출처: github.com/chrisryugj/korean-law-mcp README · 클로드 코드 `mcp add`
도움말 (2026. 9. 21. 확인)

③ 같은 질문을 다시 한다 0:45-1:15

- 1 20쪽의 질문을 토씨 하나 바꾸지 말고 다시 넣는다. 답을 `답-도구사용.md` 로 저장한다 10분
- 2 두 답을 나란히 놓고 아래 다섯 줄을 채운다 20분

무엇을 보나	도구 미사용	도구 사용
법령 이름		
조 번호		
조례를 보았나		
시행일		
숫자가 같은가		

✓ 확인 지점 — 셋째 줄에서 갈린다. 건폐율·용적률의 실제 상한은 「국토의 계획 및 이용에 관한 법률 시행령」이 정한 범위 안에서 지자체 도시계획 조례가 정한다. 조례를 보지 않은 답은 범위만 말하거나 다른 지자체의 숫자를 말한다.

도구가 있다고 늘 맞는 것은 아니다. 둘째 답도 조 번호가 틀릴 수 있다 — 그래서 다음 단계가 있다.

④ 첫 답의 인용을 검증한다 1:15-1:30

지어낸 조문을 기계로 잡는다

그대로 쓴다

"답-도구미사용.md 에 적힌 법령과 조문이 실제로 있는지 하나씩 검증해 줘.

없는 조문은 없다고 표시하고, 조문 번호는 맞는데 내용이 다르면 실제 내용을 함께 보여 줘."

korean-law 의 `verify_citations` 가 돈다. **실존 · 부존재 · 확인 불가** 셋으로 갈라 준다

✓ **확인 지점** — 첫 답에 없는 조문이 하나라도 섞여 있으면 × 로 찍힌다. 하나도 안 나와도 좋다 — 검증이 돌아갔다는 화면이 증거다.

남는 시간에 — 두 도구를 한 질문에

해 볼 것

"청주시의 부설주차장 설치기준을 조례에서 찾아 주고, 청주시 총인구를 2020년부터 연도별로 함께 보여 줘."

앞은 korean-law, 뒤는 kosis 가 받는다. **클로드가 질문을 나눠 맞는 도구로 보낸다** — 내가 어느 도구인지 말하지 않는다

과제 — 나만의 스킬을 만들어 온다

무엇을 스킬로 할지는 기록이 정한다

스킬은 결국 **프롬프트에 매번 반복해 적던 것을 대신하는 것이다**. 그러니 먼저 **내가 무엇을 반복했는지**부터 찾는다.

1 대화 기록에서 반복을 찾는다

6쪽의 프롬프트 앞에 "지난 2주 동안 클로드에서 작업한 모든 세션 기록을 검토했을 때" 를 붙여 쓴다

2 가장 값이 큰 것 하나를 고른다

세 번 넘게 한 일인가를 본다

3 `skill-creator` 로 만든다

되묻기 → 초안 → 시험 프롬프트 → 고쳐 쓰기

4 부르지 않고 꺼내지는지 확인한다

안 되면 description 을 고쳐 다시 한다

내는 것 — 분량은 세지 않는다

- **스킬 폴더 전체** — `SKILL.md` 와 그 아래 파일을 통째로
- **왜 이것을 골랐는지 한 문단** — 무엇이 몇 번 반복됐는지. 텍스트로 써도 되고 md 파일로 내도 된다
- 부르지 않았는데 **스스로 꺼내진 화면**

보는 것 셋

- 1 **반복의 근거가 있는가** — 기록에서 찾았는가
- 2 **사실이 아니라 절차인가** — 사실이면 CLAUDE.md 자리다
- 3 **스스로 꺼내지는가** — description 이 「언제 쓰는지」를 말하는가

마감 — 10월 2일

두 주를 준다. **한 주는 반복을 쌓는 기간이다** — 그동안 클로드와 일을 계속하고 그 기록에서 찾는다.

Q & A

권규상 Kyusang Kwon
kyusang.kwon@chungbuk.ac.kr